How to Create New Pickleball Lesson Handouts

This guide explains how to use the Pickleball Lesson Handouts project to create consistent student-facing PDF handouts.

Project Location

GitHub repository:

https://github.com/londonj/PickleballLessonHandouts

Local folder on the computer where this was set up:

`C:\Coding\Pickleball_Lesson_Handouts`

What This Project Does

This project creates polished PDF handouts for pickleball lessons using a consistent format.

It keeps the content and design separate:

- The lesson content is written in Markdown.
- The visual style is controlled by one shared CSS file.
- A Python script converts the Markdown into a PDF.
- Preview images are generated so the final layout can be visually checked before sharing.

This helps every handout use the same fonts, colors, section styling, spacing, and page-break rules.

Important Files and Folders

`templates/student-handout-template.md`

This is the full instructional template. It explains what each section should contain.

Use this when you need guidance on how to write the handout.

`templates/new-handout-starter.md`

This is a blank starter handout with all required section headings already included.

Use this when creating a new handout from scratch.

`templates/handout-style.css`

This controls the PDF design:

- fonts
- font sizes

- colors
- section heading bars
- margins
- spacing
- link color
- page-break behavior

Do not manually style individual PDFs. If the visual design needs to change, update this file so all future handouts stay consistent.

`scripts/build-handout-pdf.py`
This script converts a Markdown handout into a styled PDF.

It also creates preview images in the `work` folder so the PDF can be checked visually.

`scripts/compact-handout-pdf.py`
This is an optional utility that builds a much smaller version of a handout PDF (roughly one-fifth the size). It does this by replacing the color emoji in the headings with small embedded images, which avoids the large emoji data Chrome would otherwise add. The result looks the same on the page.

Use `scripts/build-handout-pdf.py` for the normal handout you share with students. Reach for this only when you specifically need a smaller file (for example, to email or attach it somewhere with a size limit).

`prompts/create-student-handout.md`
This contains the prompt to use with an AI assistant when generating a new handout.

`docs/production-checklist.md`
Use this checklist before sharing a PDF with students.

`examples/`
Completed Markdown handouts are stored here. Earlier handouts use the older `{Technique}-Student-Handout` name; new handouts use the `Lesson-Handout-{Technique}` pattern. Current examples:

- `examples/Around-The-Post-Student-Handout.md`
- `examples/Dink-Volleys-Student-Handout.md`
- `examples/Lob-Defense-Student-Handout.md`
- `examples/Overhead-Smash-Student-Handout.md`
- `examples/Third-Shot-Drop-Student-Handout.md`

`outputs/`
Finished PDF files are saved here.

`work/`
Generated HTML and preview PNG files are saved here.

This folder is for review and troubleshooting. It is ignored by Git and does not need to be shared.

Step-by-Step Workflow

Step 1: Open the Project Folder

Open this folder:

`C:\Coding\Pickleball_Lesson_Handouts`

If using a terminal:

```
cd C:\Coding\Pickleball_Lesson_Handouts
```

Step 2: Create a New Markdown Handout

Copy the starter template:

`templates/new-handout-starter.md`

Save the copy in the `examples` folder with a clear filename.

Example:

`examples/Lesson-Handout-Serving-Deep.md`

Name files using the pattern `Lesson-Handout-{Technique}` in Title-Case: start with `Lesson-Handout-`, then the technique with the first letter of each word capitalized and words separated by hyphens.

Step 3: Generate the Handout Content

Use the prompt in:

`prompts/create-student-handout.md`

The simplest way is to give an AI assistant this instruction:

```
Create a student-facing pickleball lesson handout for: [LESSON TOPIC].

Use the template at:
C:\Coding\Pickleball_Lesson_Handouts\templates\student-handout-template.md

Follow these rules:
- Use the exact numbered section headings and emojis from the template.
- Write for beginner to intermediate adult students.
- Keep the tone clear, practical, encouraging, and coach-like.
- Do not frame the handout around a lesson or a point in time. Open by describing the skill
directly (for example "The dink volley is...") rather than "This lesson covers..." or "today's
lesson," so the handout reads correctly whenever the student reviews it.
- Put all practice drills and assignments in 7. Homework 🏠.
- Put quick pre-play reminders in 10. Quick Reminders 🎗️.
- Put only outside references in 11. Resources 🔗.
- Do not duplicate drills or review lists in Resources.
- Use current web research for Resources.
- Include exactly three article links from three different sources.
- Include exactly three YouTube links from three different creators/channels.
- Include useful rule or strategy references only when they support the lesson.
- A numbered section must never split across pages. The build keeps each section together
automatically; if a section is too tall to fit on one page, shorten it so it fits.
- Always visually inspect the rendered preview PNGs after building, and confirm no section is split
across pages. Never rely on page count alone.
- Name the files using the pattern Lesson-Handout-[Technique] in Title-Case with dashes (for
example Lesson-Handout-Overhead-Smash). Save the finished Markdown handout in:
  C:\Coding\Pickleball_Lesson_Handouts\examples\Lesson-Handout-[Technique].md

After writing the Markdown, build the PDF using:
python C:\Coding\Pickleball_Lesson_Handouts\scripts\build-handout-pdf.py
C:\Coding\Pickleball_Lesson_Handouts\examples\Lesson-Handout-[Technique].md -o
C:\Coding\Pickleball_Lesson_Handouts\outputs\Lesson-Handout-[Technique].pdf

Before delivering the PDF, visually inspect the generated preview PNGs in:
C:\Coding\Pickleball_Lesson_Handouts\work\lesson-handout-[technique]-preview

Confirm that major sections do not break awkwardly across pages and that Resources is present and
coherent.
```

Replace `[LESSON TOPIC]` with the actual lesson topic.

Replace `[Technique]` with the topic in Title-Case: capitalize the first letter of each word and separate words with hyphens. The full filename base is then `Lesson-Handout-[Technique]`.

Example:

- Lesson topic: `Deep Serve and Return`
- Technique: `Deep-Serve-And-Return`
- Filename base: `Lesson-Handout-Deep-Serve-And-Return`
- Markdown file: `examples/Lesson-Handout-Deep-Serve-And-Return.md`
- PDF file: `outputs/Lesson-Handout-Deep-Serve-And-Return.pdf`

Step 4: Build the PDF

From the project folder, run:

```
python .\scripts\build-handout-pdf.py .\examples\Lesson-Handout-[Technique].md -o .\outputs\Lesson-Handout-[Technique].pdf
```

Example:

```
python .\scripts\build-handout-pdf.py .\examples\Lesson-Handout-Serving-Deep.md -o .\outputs\Lesson-Handout-Serving-Deep.pdf
```

By default, no section is forced onto a new page. Each section is kept together and sections flow to fill each page, moving to the next page only when they will not fit as a whole.

Step 5: Visually Check the PDF

After the script runs, check the generated preview images in:

`work\lesson-handout-[technique]-preview`

Open each `page-#.png` image and check:

- Section headings are not stranded at the bottom of a page.
- No numbered section is split across pages. Each section sits wholly on one page.
- Pages are reasonably filled; no large white-space gap is left where more sections could have fit.
- Resources is present and complete.
- Text is not clipped, overlapped, or too small.
- Link titles display correctly.
- The document feels polished and readable.

Do not rely only on page count or text extraction. Always visually inspect the preview images.

Step 6: Review the Content

Use:

`docs/production-checklist.md`

Confirm:

- All 11 sections are present.
- The lesson topic is clear.
- Homework includes the drills.
- Quick Reminders includes only short pre-play cues.
- Resources includes only outside references.
- Resources has exactly three articles and exactly three YouTube videos.
- Article sources are not repeated.
- YouTube creators/channels are not repeated.
- Rule or strategy references are useful and relevant.

Step 7: Share the PDF

Use the finished PDF from:

`outputs\`

Example:

`outputs\Lesson-Handout-Serving-Deep.pdf`

Common Commands

Build a new handout:

```
python .\scripts\build-handout-pdf.py .\examples\Lesson-Handout-[Technique].md -o .\outputs\Lesson-Handout-[Technique].pdf
```

Example, building the overhead smash handout:

```
python .\scripts\build-handout-pdf.py .\examples\Lesson-Handout-Overhead-Smash.md -o .\outputs\Lesson-Handout-Overhead-Smash.pdf
```

Force Resources to start on a new page:

```
python .\scripts\build-handout-pdf.py .\examples\Lesson-Handout-[Technique].md -o .\outputs\Lesson-Handout-[Technique].pdf --page-start-sections 11
```

Force sections 7, 8, and 11 to start on new pages:

```
python .\scripts\build-handout-pdf.py .\examples\Lesson-Handout-[Technique].md -o .\outputs\Lesson-Handout-[Technique].pdf --page-start-sections 7,8,11
```

Setup Requirements

This project needs:

- Python 3.10 or newer
- Google Chrome or Microsoft Edge
- Python packages listed in `requirements.txt`

Install the Python packages from inside the project folder:

```
python -m pip install -r requirements.txt
```

If the script cannot create preview images, install the requirements again:

```
python -m pip install pypdf pymupdf
```

Caveats and Special Notes

- Name handout files using the pattern `Lesson-Handout-{Technique}` in Title-Case with hyphens (for example `Lesson-Handout-Overhead-Smash.pdf`). Use the same name for the Markdown and the PDF.

- Resources must be researched each time because links, article availability, YouTube titles, and view counts can change.
- The AI assistant should browse the web when creating the Resources section.
- The Resources section should not include practice drills or repeated review lists.
- Do not edit the PDF directly.
- Do not manually change fonts, colors, or spacing inside individual handouts.
- If the visual style needs to change, edit `templates/handout-style.css`.
- If the handout structure needs to change, edit `templates/student-handout-template.md`.
- If page breaks look awkward, rebuild with a different `--page-start-sections` value.
- Always inspect the rendered PNG previews before sending the PDF to students.
- The `work` folder is generated output and is intentionally ignored by Git.

Publishing Changes to GitHub

After adding or updating handouts, commit and push the changes. Always push after committing so the GitHub copy stays in sync:

```
git status
git add .
git commit -m "Add [lesson topic] handout"
git push
```

Use a clear commit message, such as:

```
git commit -m "Add deep serve handout"
```

Quick Summary

1. Copy `templates/new-handout-starter.md`.
2. Save it in `examples/` with a lesson-specific filename.
3. Use `prompts/create-student-handout.md` to generate the content.
4. Build the PDF with `scripts/build-handout-pdf.py`.
5. Check the preview PNGs in `work/`.
6. Review with `docs/production-checklist.md`.
7. Share the PDF from `outputs/`.